

# claude-analytic

*See where your Claude Code tokens actually went.*

Tokens ran out faster than expected? Don't know where they went? This tool shows you — locally, in seconds. claude-analytic is a CLI that reads your local Claude Code sessions and reports token usage, cache hit rate, and daily costs so you can see why your quota disappeared.

## Quick Start

Run it instantly with npx — no install needed:

```
npx github:onsaurav/claude-analytic
```

Or install it globally:

```
npm install -g claude-analytic  
claude-analytic
```

## Key Concepts

### What is a Token?

A token is the smallest unit of text Claude reads or writes — roughly  $\frac{3}{4}$  of a word, or about 4 characters in English. Every prompt you send and every reply Claude returns is measured in tokens, and that is what you are billed for.

- "Hello, world!" → about 4 tokens
- A 500-word message → about ~650 tokens
- A medium source file → 1,000–3,000 tokens

There are four kinds of tokens this tool tracks:

| Token Type  | What It Means                                                                             |
|-------------|-------------------------------------------------------------------------------------------|
| Input       | Fresh text you sent to Claude (not cached).                                               |
| Output      | Text Claude generated back to you. 5× more expensive than input.                          |
| Cache write | Input that was stored into the cache for reuse.                                           |
| Cache read  | Input served from the cache instead of being re-processed — 10× cheaper than fresh input. |

## What is a Cache Hit?

When you keep talking in the same session, Claude doesn't re-read your earlier messages from scratch — it pulls them from a prompt cache. Each time the cache is reused, that's a cache hit.

- **Cache hit** → input served from cache → costs \$0.30 / 1M tokens
- **Cache miss** → input processed fresh → costs \$3.00 / 1M tokens (10× more)

**Cache hit rate** =  $\text{cache reads} \div (\text{cache reads} + \text{fresh input})$ . A higher rate means you're reusing context efficiently and paying less.

## How It Works

- Claude Code stores every session locally as JSONL files under `~/.claude/projects/`.
- This tool reads those files — no network calls, no API keys needed.
- It tallies up token counts per session and per day, applies the published pricing, and prints a summary.
- Done. Everything stays on your machine.

## Why You'd Use It

Sometimes your tokens vanish way before you expected. The usual suspects:

| What Happened                      | Why Tokens Drained                                        |
|------------------------------------|-----------------------------------------------------------|
| Big code generation / refactor     | Output tokens cost 5× input — they add up fast.           |
| You ran <code>/clear</code> often  | Each clear kills the cache → next prompts pay full price. |
| Hopping between many projects      | Each project has its own cache. None stay warm.           |
| Long idle gaps between turns       | Cache expires → you pay the warm-up again.                |
| Edited a file Claude read early on | Prefix changed → everything after it cache-misses.        |

**Cache hit rate** is the number that tells the story: cache reads cost \$0.30 / 1M vs \$3.00 / 1M for fresh input — 10× cheaper. A day at 90% hit vs 30% can be a 3–5× cost difference.

| Hit Rate | Verdict                                                            |
|----------|--------------------------------------------------------------------|
| 85%+     | <span style="color: green;">●</span> Excellent                     |
| 60–85%   | <span style="color: yellow;">●</span> Normal — room to improve     |
| < 50%    | <span style="color: red;">●</span> Paying full price on most input |

## What It Shows

When you run it, you get three sections — example output:

Claude Analytic Report

Detected Claude path: C:\Users\you\.claude\projects  
Total sessions: 3

► Overall Summary

| Metric                   | Value     |
|--------------------------|-----------|
| Total Input Tokens       | 272       |
| Total Output Tokens      | 62,260    |
| Total Cache Read Tokens  | 4,123,997 |
| Total Cache Write Tokens | 135,468   |
| Overall Cache Hit Rate   | 99.99%    |

► Daily Costs

| Date       | In  | Out    | Cache R   | Cache W | Cost     |
|------------|-----|--------|-----------|---------|----------|
| 2026-05-01 | 272 | 62,260 | 4,123,997 | 135,468 | \$2.6799 |

Total cost across all days: \$2.6799

► Top 5 Sessions by Token Usage

| Session                | In  | Out    | Cache Hit |
|------------------------|-----|--------|-----------|
| d--workspace-claude... | 153 | 28,150 | 99.99%    |
| d--workspace-c160c1... | 69  | 21,284 | 99.99%    |
| d--workspace-claude... | 50  | 12,826 | 99.99%    |

💡 Tip: High cache hit rate = lower cost

## What Each Section Tells You

- **Overall Summary** — totals for every token type and your overall cache hit rate. Green = healthy.
- **Daily Costs** — one row per day, so you can spot the expensive days at a glance.
- **Top 5 Sessions by Token Usage** — your biggest spenders, ranked, with their individual hit rate.

## Pricing Used

| Token Type  | Rate per 1M Tokens |
|-------------|--------------------|
| Input       | \$3.00             |
| Output      | \$15.00            |
| Cache read  | \$0.30             |
| Cache write | \$3.75             |

## Tips to Save Tokens

- **Stay in one session** — every `/clear` is a cold cache.
- **Put stable context first** — the cache matches a prefix.
- **Don't ping-pong between projects** in short bursts.

## Privacy

**Everything runs locally.** No data leaves your machine — no network calls, no telemetry, no API keys.